1. Introduction

Deadlock is a situation in an operating system where two or more processes wait for resources held by each other. Due to this situation, none of the processes can continue execution.

Deadlock management is an important concept in operating systems because it helps in proper resource allocation and smooth execution of processes.

This project, Deadlock Detection and Avoidance Simulator, is developed using Python. It demonstrates how deadlocks can be detected and avoided using Banker's Algorithm and the Deadlock Detection Algorithm.

2. Abstract

The Deadlock Detection and Avoidance Simulator is a Python-based project that demonstrates resource allocation and deadlock management in an operating system.

The project implements two important algorithms: Banker's Algorithm and Deadlock Detection Algorithm. Banker's Algorithm checks whether the system is in a safe state before granting resources, while the Deadlock Detection Algorithm identifies processes that cannot complete due to resource unavailability.

The simulator accepts Allocation, Maximum, Available and Request matrices from the user. It calculates the Need Matrix, checks the safe sequence and displays the final result.

3. Objectives

The main objectives of this project are:

1. To understand the concept of deadlock in operating systems.
2. To implement Banker's Algorithm using Python.
3. To implement the Deadlock Detection Algorithm.
4. To calculate the Need Matrix.
5. To identify safe and unsafe system states.
6. To display the safe sequence of processes.
7. To check additional resource requests.

4. Deadlock in Operating Systems

Deadlock occurs when multiple processes are unable to proceed because each process is waiting for a resource held by another process.

Example:

- Process P1 holds Resource R1 and waits for R2.
- Process P2 holds Resource R2 and waits for R1.
- Both processes wait for each other.

As a result, neither process can continue execution.

Four Necessary Conditions of Deadlock

Deadlock can occur when all four conditions are present:

1. Mutual Exclusion: A resource can be used by only one process at a time.
2. Hold and Wait: A process holds one resource while waiting for another.
3. No Preemption: Resources cannot be forcibly taken from a process.
4. Circular Wait: Processes wait for resources in a circular chain.

5. Banker's Algorithm

Banker's Algorithm is a deadlock avoidance algorithm used by operating systems. It checks whether allocating resources to a process will keep the system in a safe state.

Before granting a resource request, the algorithm checks whether all processes can complete their execution in some possible order.

Data Structures Used

- Allocation Matrix: Stores the resources currently allocated to each process.
- Maximum Matrix: Stores the maximum resources required by each process.
- Available Vector: Stores the number of currently available resources.
- Need Matrix: Stores the remaining resources required by each process.

Need Matrix Formula

\[Need = Maximum - Allocation \]

Working Steps

1. Calculate the Need Matrix.
2. Initialize Work with Available resources.
3. Find a process whose Need is less than or equal to Work.
4. Assume that the process completes its execution.
5. Add its allocated resources back to Work.
6. Repeat the process until all processes finish or no process can proceed.
7. If all processes finish, the system is safe; otherwise, it is unsafe.

6. Deadlock Detection Algorithm

The Deadlock Detection Algorithm is used to identify whether a deadlock currently exists in the system.

It uses three main data structures:

- Allocation Matrix

- Request Matrix
- Available Vector

Working Steps

1. Initialize Work with Available resources.
2. Mark processes with zero allocated resources as finished.
3. Find an unfinished process whose Request can be satisfied by Work.
4. Assume that the process completes and releases its resources.
5. Add the released resources to Work.
6. Repeat until no more processes can finish.
7. If any processes remain unfinished, deadlock is detected.

System Workflow

User Input → Matrix Creation → Need Calculation → Banker's Algorithm → Deadlock Detection → Resource Request Check → Final Result

7. Software and Hardware Requirements

Software Requirements

Component	Requirement
Operating System	Windows / Linux
Programming Language	Python 3
IDE	VS Code / IDLE
Libraries	No external libraries

Hardware Requirements

Component	Requirement
Processor	Dual Core or above
RAM	Minimum 4 GB
Storage	100 MB free space

8. Functions Used in the Project

Function	Purpose
`read_matrix()`	Accepts matrix input
`need_matrix()`	Calculates remaining resource needs
`print_matrix()`	Displays matrices
`safety_algorithm()`	Checks system safety
`detect_deadlock()`	Detects deadlock
`resource_request()`	Checks additional resource requests
`main()`	Executes the complete program

8. Python Source Code

def read_matrix(n, m, label): print(f"Enter {label} matrix ({n} rows, {m} values each):") data = []

```
for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))

    if len(row) != m or any(x < 0 for x in row):
        raise ValueError("Invalid matrix input")

    data.append(row)

return data
```

def need_matrix(maximum, allocation): return [[maximum[i][j] - allocation[i][j] for j in range(len(maximum[0]))] for i in range(len(maximum))]

def print_matrix(title, matrix): print("\n" + title)

```
for i, row in enumerate(matrix):
    print(f"P{i}: " + " ".join(map(str, row)))
```

def safety_algorithm(allocation, maximum, available, show=True): n, m = len(allocation), len(available)

```
need = need_matrix(maximum, allocation)
work = available.copy()
finish = [False] * n
```

```
sequence = []

if show:
    print_matrix("Need Matrix", need)
    print("Initial Work:", work)

while True:
    progress = False

    for i in range(n):
        if not finish[i] and all(
            need[i][j] <= work[j] for j in range(m)
        ):
            if show:
                print(f"P{i} can finish")

            work = [
                work[j] + allocation[i][j]
                for j in range(m)
            ]

            finish[i] = True
            sequence.append(i)
            progress = True

            if show:
                print(f"After P{i}, Work = {work}")

    if not progress:
        break

safe = all(finish)

if show:
    if safe:
        print("\nSAFE STATE")
        print("Safe Sequence:",
              " -> ".join(f"P{i}" for i in sequence))
    else:
        print("\nUNSAFE STATE")
        print("Unfinished processes:",
              [f"P{i}" for i in range(n) if not finish[i]])

return safe, sequence, need


def detect_deadlock(allocation, request, available): n, m = len(allocation), len(available)

work = available.copy()

finish = [
    all(x == 0 for x in allocation[i])
    for i in range(n)
]

sequence = []

print("\nDeadlock Detection")
print("Initial Work:", work)

while True:
    progress = False

    for i in range(n):
        if not finish[i] and all(
```

```
            request[i][j] <= work[j] for j in range(m)
        ):
            print(f"P{i}: Request can be satisfied.")

            work = [
                work[j] + allocation[i][j]
                for j in range(m)
            ]

            finish[i] = True
            sequence.append(i)

            print(f"P{i} finishes; Work = {work}")
            progress = True

    if not progress:
        break

remaining = [
    f"P{i}" for i in range(n) if not finish[i]
]

if remaining:
    print("DEADLOCK DETECTED:", remaining)
else:
    print("NO DEADLOCK DETECTED")
    print("Completion order:",
          " -> ".join(f"P{i}" for i in sequence) or "None")

return remaining


def resource_request(pid, request, allocation, maximum, available): n, m = len(allocation), len(available)

if pid < 0 or pid >= n:
    print("Invalid process number.")
    return allocation, available

need = need_matrix(maximum, allocation)

if any(request[j] > need[pid][j] for j in range(m)):
    print("DENIED: request exceeds remaining Need.")
    return allocation, available

if any(request[j] > available[j] for j in range(m)):
    print("WAIT: resources are unavailable.")
    return allocation, available

new_a = [row.copy() for row in allocation]
new_v = available.copy()

for j in range(m):
    new_a[pid][j] += request[j]
    new_v[j] -= request[j]

safe, seq, _ = safety_algorithm(
    new_a, maximum, new_v, show=False
)

if safe:
    print("REQUEST GRANTED; system remains safe.")
    print("Safe Sequence:",
          " -> ".join(f"P{i}" for i in seq))

    return new_a, new_v
```

```
print("REQUEST DENIED; state would become unsafe.")
return allocation, available


def main(): print("DEADLOCK DETECTION & AVOIDANCE SIMULATOR")

try:
    n = int(input("Number of processes: "))
    m = int(input("Number of resource types: "))

    if n <= 0 or m <= 0:
        raise ValueError("Counts must be positive.")

    allocation = read_matrix(n, m, "Allocation")
    maximum = read_matrix(n, m, "Maximum")

    available = list(map(
        int, input(f"Available vector ({m} values): ").split()
    ))

    if len(available) != m or any(x < 0 for x in available):
        raise ValueError("Invalid Available vector.")

    for i in range(n):
        for j in range(m):
            if allocation[i][j] > maximum[i][j]:
                raise ValueError("Allocation exceeds Maximum.")

    safety_algorithm(allocation, maximum, available)

    print("\nEnter current outstanding Request matrix.")
    request = read_matrix(n, m, "Request")

    detect_deadlock(allocation, request, available)

    choice = input(
        "\nCheck an additional resource request? (y/n): "
    ).strip().lower()

    if choice == "y":
        pid = int(input(f"Process number (0 to {n-1}): "))

        req = list(map(
            int, input(f"Request vector ({m} values): ").split()
        ))

        if len(req) != m or any(x < 0 for x in req):
            raise ValueError("Invalid request vector.")

        resource_request(
            pid, req, allocation, maximum, available
        )

except ValueError as e:
    print("Input Error:", e)


if name == "main": main()
```

10. Sample Input

For testing, the simulator uses 5 processes and 3 resource types.

Allocation Matrix

```
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2
```

Maximum Matrix

```
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3
```

Available Vector

```
3 3 2
```

11. Sample Output

```
Need Matrix

P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

Banker's Algorithm Output

SAFE STATE

Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2

Deadlock Detection Output

For this example, the outstanding Request Matrix contains zero requests
for all processes.

NO DEADLOCK DETECTED

Additional Resource Request

Process: P1
Request: 1 0 2
```

```
REQUEST GRANTED
System remains safe.
```

12. Working and Performance Analysis

The simulator first accepts the number of processes and resource types. It then takes the Allocation Matrix, Maximum Matrix and Available Vector as input.

After receiving the input, it calculates the Need Matrix using the formula:

\[Need = Maximum - Allocation \]

Banker's Algorithm checks whether the system can execute all processes in a safe sequence. After that, the Deadlock Detection Algorithm checks the current outstanding resource requests.

For the sample input, the safe sequence is:

P1 → P3 → P4 → P0 → P2

All processes can finish, so the system is in a safe state.

Time Complexity:

Operation	Complexity
Need Matrix	$O(n \times m)$
Banker's Algorithm	$O(n^2 \times m)$
Deadlock Detection	$O(n^2 \times m)$
Resource Request	$O(n^2 \times m)$

Here, `n` represents the number of processes and `m` represents the number of resource types.

13. Result

The project successfully implements Banker's Algorithm and the Deadlock Detection Algorithm using Python.

It calculates the Need Matrix, identifies safe sequences, detects deadlocks and checks additional resource requests.

For the given sample input, the system is in a safe state and no deadlock is detected.

14. Conclusion

The Deadlock Detection and Avoidance Simulator helps in understanding how operating systems manage resources among multiple processes.

Through this project, Banker's Algorithm and the Deadlock Detection Algorithm were implemented successfully using Python. The project provides a practical understanding of deadlock, resource allocation, safe sequences and system safety.

15. References

1. Operating System Concepts – Abraham Silberschatz, Peter B. Galvin and Greg Gagne.
2. Python Official Documentation – https://docs.python.org/3/
3. GeeksforGeeks – Banker's Algorithm.
4. GeeksforGeeks – Deadlock Detection Algorithm.